

Kocaeli Üniversitesi Bilgisayar Mühendisliği

PROGRAMLAMA LABORATUVARI-I

PROJE - I RAPORU

Senanur Demirci

Projede Java programlama dili kullanılmıştır.

İÇİNDEKİLER

Programın Amacı

Çözüm Tasarımı

-Kullanıcıya Yönelik Tasarım

-Programlamaya Yönelik Tasarım

Sonuçların Değerlendirilmesi

Kaynaklar

I. PROGRAMIN AMACI

Belirli kurallara göre hareket eden bir robotun önündeki engelleri aşarak istenen hedefe ulaşmasını sağlayan bir oyun tasarlamaktır. Oyunda iki adet problemin çözülmesi gerekmektedir.

Problem 1: Bu problemde robot url üzerinden veri çekilerek oluşan ızgara üzerinde verilen hedefe engellere takılmadan en kısa sürede ve en kısa yoldan ulaşmalıdır. Robot tüm ızgarayı bilmez, yalnızca birkaç kare ilerisini görebilmektedir.

Problem 2: Bu problemde robot rastgele oluşan bir labirent üzerinde ilerleyerek çıkışa ulaşmalıdır. Labirent üzerinde çıkışı olmayan yollar bulunmaktadır ve robot tüm ızgarayı bilmediğinden bu yollara girebilir. Bu durumda geri dönüp çıkışı aramaya devam etmeli ve sonunda bulmalıdır.

II. ÇÖZÜM TASARIMI

A. KULLANICIYA YÖNELİK TASARIM:

Program çalıştırıldığında kullanıcının karşısına bir panel çıkacak ve bu panelde hangi problemi çözmek istediğine dair seçim yapması için butonlar bulunacaktır. Kullanıcının seçimi üzerine;

1) Problem 1'i seçerse: Üzerinde "Çalıştır", "URL Değiştir" ve "Sonuç Göster" yazan butonların ve url'den alınan bilgilere göre oluşturulmuş labirentin bulutlanmış hali bulunan bir ekran açılacaktır.

Kullanıcının "Çalıştır" butonuna basması durumunda program çalışacak ve labirent çözülecektir.

Kullanıcının "URL Değiştir" butonuna basması durumunda url değişecek ve labirent yeni url'den alınan verilere göre yeniden oluşacaktır.

Kullanıcının "Sonuç Göster" butonuna basması durumunda yeni bir panel açılacak ve labirentin çözülmüş hali kullanıcıya gösterilecektir.

2) Problem 2'yi seçerse: Bir mesaj kutuğu açılacak ve kullanıcıdan oluşturmak istediği labirentin boyutunun girilmesi istenecektir. Labirent boyutu girildiğinde ise, üzerinde "Çalıştır", "Labirent Değiştir" ve "Sonuç Göster" yazan butonların ve rastgele oluşturulmuş labirentin bulutlanmış hali bulunan bir ekran açılacaktır.

Kullanıcının "Çalıştır" butonuna basması durumunda program çalışacak ve labirent çözülecektir.

Kullanıcının "Labirent Değiştir" butonuna basması durumunda kullanıcıdan tekrardan labirentin boyutu istenecek ve girilen değere göre labirent oluşturulacaktır.

Kullanıcının "Sonuç Göster" butonuna basması durumunda yeni bir panel açılacak ve labirentin çözülmüş hali kullanıcıya gösterilecektir.

B. PROGRAMLAMAYA YÖNELİK TASARIM:

Öncelikle projede bulunan sınıflardan bahsetmeliyiz.

Sınıfların isimleri: Proje, Izgara, Uygulama, Engel, Robot,

1) *Proje sınıfı:* main'i içeren sınıftır. JFrame'den kalıtım alır. Program çalışınca kullanıcının karşısına çıkan ilk arayüz ekranı burada oluşur.

2) *Izgara sınıfı:* Izgaraların oluştuğu sınıftır. JFrame'den kalıtım alır. problem1, problem1-izgara ve problem2 isimli üç methodu bulunmaktadır. Parametrelili ve parametresiz olmak üzere iki yapıcı methodu vardır.

3) *Uygulama sınıfı:* Problemlerin labirent çözümleri bu sınıfta yapılır. JFrame'den kalıtım alır. Parametrelili ve parametresiz olmak üzere iki yapıcı methodu vardır. İçerisinde problem2coz, cozum1, cozum2 ve bulutlama isminde methodlar bulunur. Problem1-Cozum sınıfına bu sınıf üzerinden ulaşılır. Ayrıca txt dosyasına yazma işlemleri bu sınıfta yapılır.

4) *Problem1-Cozum sınıfı:* Problem 1'in labirent çözümü bu sınıfta yapılmaktadır. Parametrelili ve parametresiz olmak üzere iki yapıcı methodu vardır.

5) *Engel sınıfı:* Farklı parametreler içeren üç yapıcı methodu vardır. JFrame'den kalıtım alır. Arayuz1 ve Arayuz2 sınıflarına bu sınıf üzerinden ulaşılır.

6) *Arayuz1 sınıfı:* Farklı parametreler içeren iki yapıcı methodu vardır. JFrame'den kalıtım alır. Problem 1'in labirentinin görsel arayüzü burada oluşur.

7) *Arayuz2 sınıfı:* Farklı parametreler içeren iki yapıcı methodu vardır. JFrame'den kalıtım alır. Problem 2'in labirentinin görsel arayüzü burada oluşur.

8) *p1gorsel sınıfı:* Kullanıcının başlangıçta yaptığı seçimin Problem 1 olması durumunda açılan panelin oluştuğu sınıftır. JFrame'den kalıtım alır. Parametrelili ve parametresiz olmak üzere iki yapıcı methodu vardır.

9) *Robot sınıfı:* Problem 2 için, robotun labirentte bulunduğu yeri tutan ve Arayuz2 sınıfına gönderip robotun görsel arayüzde oluşmasını sağlayan sınıftır.

C. PROJENİN ÇALIŞMASI

Proje çalıştırıldığında ekranda bir panel açılacak ve kullanıcının seçim yapması istenecektir.

1) *PROBLEM 1'İ SEÇERSE:* Kullanıcı 'Problem 1' isimli butona bastığında buton için actionPerformed methodu çalışır ve bu methodun içerisinde öncelikle Izgara türünde bir nesne oluşturulur. Bu nesne kullanılarak Izgara sınıfında bulunan problem1 isimli methoda gidilir. Bu method parametrelili bir methoddur ve bu parametre integer bir değerdir. Başlangıçta butondan değer olarak '1' gelmektedir. Bu sayede hangi url'den değer alınması gerektiğinin kontrolü yapılır. Dosya yolu gerekli satıra kopyalanarak url'den veri alınması işlemi gerçekleştirilir. Url'den alınan veriler her satır bir string olacak şekilde bir değişkende tutulur ve daha sonra integera çevrilir. Integera dönüştürülen satırları tek tek rakamlara çevirmek için belirli matematiksel işlemler yapılır ve bu matris ham-matris isimli bir matrise atanır. Ancak ham-matris istediğimiz matrisin simetrik olarak tersi durumunda bulunduğundan, ham-matrisin simetriği alınarak matris isimli iki boyutlu bir diziye atanır. Bu matris problem1-izgara isimli methoda gönderilir.

problem1-izgara methodunda öncelikle gerekli değişken atamaları yapılır. Matris üzerinde rastgele giriş-çıkış seçimi sırasında kolaylık olması için tüm matrisin dışı 9 larla kaplanır. 4 kenardan biri üzerinde rastgele bir duvar giriş olarak seçilir ve aynı duvar üzerinde olmama şartı ile rastgele bir çıkış seçilir. Giriş indeksine 7; çıkış indeksine 6 değeri atanır. Matris konsol ekranına yazdırılır. Daha sonra görsel olarak oluşması için p1gorsel sınıfına gerekli parametrelerle gönderilir.

p1gorsel sınıfı, url'den veri alınarak oluşturulan matrisin bulutlanmış halinin ve "Çalıştır", "URL değiştir", "Sonuç Göster" butonlarının bulunduğu panel ekranının görsel olarak oluştuğu sınıftır.

"Çalıştır" butonuna basılması durumunda actionPerformed methodu çalışır ve Uygulama türünde bir nesne oluşarak Uygulama sınıfına gerekli parametreler gönderilir. Uygulama sınıfının yapıcı methodunun çalışması sonucu Problem1-Cozum sınıfının yapıcı methodu çalışır. Daha sonra Cozum isimli method çalışarak labirentin çözümünü stack yapısını kullanarak çözer. İlerleme methodu çalışır ve robot ilerledikçe çevresinin bulutunun kalkmasını sağlar. Daha sonra Engel sınıfından bir nesne oluşturularak engel sınıfındaki yapıcı methodun çalışması sağlanır.

Ayrıca bu sınıfta robotun bulunduğu indeksler adım adım Dosya1 isimli ve .txt uzantılı bir dosyaya kaydedilir. "Çalıştır" butonuna her basıldığında bu dosya silinir ve yeniden oluşur.

Engel sınıfının yapıcı methodundan Arayuz1 isimli sınıfa ulaşılır. Burada çözümün görsel arayüzü oluşmaktadır. paint methodu Override edilir ve engellere, robota duvarlara, robotun ilerlemesine, giriş ve çıkışa renk atamaları gerçekleştirilir.

"URL değiştir" butonuna basılması durumunda actionPerformed methodu çalışır ve sayac kontrolü ile problem1 methoduna bu defa '2' değeri gönderilir. Farklı bir urlden veri alınır ve problem1-izgara isimli methoda gidilir. Yukarıda anlatılan işlemler adım adım tekrarlanır ve plgorsel sınıfına gidilir ve oradan yapılacak buton seçimine göre program çalışmaya devam eder.

"Sonuç Göster" butonuna basılması durumunda actionPerformed methodu çalışır ve Uygulama sınıfına, Uygulama sınıfından Problem1-Cozum sınıfına gidilir. if bloğu kontrolü ile Engel sınıfının farklı bir yapıcı methoduna ulaşılır. Bu yapıcı methodun çalışması ile de Arayuz1 sınıfının farklı bir yapıcı methoduna ulaşılır. Arayuz1 sınıfında paint methodu Override edilir. Çözümün yalnızca robotun çıkışa ulaştığı hali ekranın ortasında bir panel açılarak kullanıcıya gösterilir.

2) **PROBLEM 2'Yİ SEÇERSE:** Kullanıcı 'Problem 2' isimli butona bastığında buton için actionPerformed methodu çalışır ve bir dialog paneli açılır. Kullanıcıdan oluşturulacak matris için boyut girilmesi istenir. Izgara türünde bir nesne oluşturularak Izgara sınıfının yapıcı methoduna girilen matris değeri gönderilir. Yapıcı method çalışır ve problem2 isimli methoda gidilir. Burada yeni bir görsel panel oluşturulur ve içerisinde "Çalıştır", "Labirent Değiştir", "Sonuç Göster" butonlarını ve aynı zamanda oluşan labirentin bulutlu görüntüsünü bulundurur.

"Çalıştır" butonuna basılması durumunda actionPerformed methodu çalışır ve Uygulama sınıfındaki problem2coz methodu çalışır. Burada labirent rastgele atamalarla 1 veya 0 lardan oluşturulur. Daha sonra üst, alt, sağ ve sol duvarlara 1 ataması yapılır. Rastgele seçilen girişe göre çapraz köşe çıkış olarak belirlenir. Giriş ve çıkışa göre cozum1 veya cozum2 methodlarına gidilir.

cozum1 methodunda girişten çıkışa kadar gitmesi gerektiği sayıda sağ ve alt isimli stringler yon isimli bir ArrayListe eklenir. Başlangıç indeksleri cozum isimli hucre dizisine eklenir. hucre bir sınıftır ve içerisinde satır ve sütun özellikleri bulundurur. Daha sonra bir while döngüsüne girilir. Bu döngü yon ArrayListinin içinde eleman bulunduğu sürece çalışacaktır. yon ArrayListi içerisinde rastgele bir yön seçilir ve seçilen yöne göre yoldevam isimli bir diziye gidilen yönün indeksleri yazılır. Sonra dizinin bir sonraki elemanına geçilir. İşlem tekrarlanır. Her yeni elemana geçmeden önce yon ArrayListinden gidilen yön tekrar seçilmemesi için çıkartılır. Matris konsola yazdırılır. Daha sonra cozum dizisine çıkış indekslerinin değerleri atanır. Gidilen bütün indeksler sırasıyla 'Dosya2' isimli .txt uzantılı bir dosyaya kaydedilir. Ayrıca 'Çalıştır' butonuna her basıldığında bu dosya silinerek yeniden oluşur.

Daha sonra başlangıçta tamamı 9'dan oluşan bir matris oluşturulur. For döngüleriyle robotun satır ve sütununu cozum dizisinden teker teker alarak, robotun ilerlemesine

göre bulutlama kaldırılır. Robot sınıfı türünde bir nesne oluşturularak Robot sınıfının yapıcı methoduna gidilir. Robotun bulunduğu indekslere konsolda 'R' harfi eklenir. Arayuz2 sınıfının yapıcı methoduna gidilir. paint methodu Override edilir ve çözümün görsel arayüzü burada oluşturulur. Robot çıkmaz yollara girdiğinde kırmızı renge dönüşerek o noktanın çıkmaz yol olduğu belirtilir ve geri döner. Her indeks değişikliğinde robot sınıfının yapıcı methoduna gidilir ve Arayuz2 de oluşan görsel arayüzde robot ilerletilmiş olur. Ayrıca görsel arayüz panelinde çıkışa ulaşmak için gidilen hamle sayısı da gösterilir.

cozum2 methodunda girişten çıkışa kadar gitmesi gerektiği sayıda sağ ve ust isimli stringler yon isimli ArrayListe eklenir ve cozum1'deki adımlar uygulanır.

"Labirent Değiştir" butonuna basılması durumunda actionPerformed methodu çalışır ve kullanıcıdan matris boyutu istenir. Girilen değer Izgara sınıfının yapıcı methoduna gönderilir. Daha sonra problem2 methoduna gidilir ve işlemler adım adım gerçekleşir.

"Sonuç Göster" butonuna basılması durumunda actionPerformed methodu çalışır ve Engel sınıfı türünde bir nesne oluşturularak Engel sınıfının yapıcı methoduna gidilir. Engel sınıfından Arayuz2 sınıfına ulaşılır. Arayuz2 sınıfında paint methodu Override edilir. Çözümün yalnızca robotun çıkışa ulaştığı hali ekranın ortasında bir panel açılarak kullanıcıya gösterilir.

D. SONUÇLARIN DEĞERLENDİRİLMESİ

1) *Programdaki Eksiklikler ve Hatalar:* - Robot adım adım ilerlemiyor.

- Çalıştır butonuna basılınca her adımda yeni arayüz ekranı açılıyor.

- Programın çalışma süresi ekranda gösterilmiyor.

E. KAYNAKLAR

- <https://tugrulbayrak.medium.com/uml-class-diyagramlari-4c3bb7e9cc4c>

- <https://serdarkuzucu.com/javada-timer-ve-timertask-siniflarinin-kullanimi/>

- <https://kodlamavakti.com/java/jbutton-kullanimi-ve-butona-basma-olaylari/>

- <https://thecodeprogram.com/java-ile-stack-sinifi-ve-kullanimi>

- <http://www.migapro.com/depth-first-search/>

- <https://www.geeksforgeeks.org/stack-class-in-java/>

- <https://www.youtube.com/watch?v=dyrvXiMumXc>

- <https://www.youtube.com/watch?v=W9F8fDQj7Ok>

